

One Integer for Work and Memory: A Mechanized, Content-Addressed Combinator Machine, and the Engineering of Trustworthy Proof Artifacts

true

2026-07-30

Contents

Abstract	2
1. Introduction	2
2. Background: what “checkable” has to mean	4
3. The machine	4
3.1 Nodes, bytes, identity	4
3.2 Terms, thunks, and the size model	5
3.3 Reduction and pricing	5
3.4 The bound	6
3.5 Canonical compilation	6
4. The mechanization	6
4.1 Bytes (<code>Sha256.lean</code> , <code>MachineBytes.lean</code>)	6
4.2 The evaluator (<code>EvalMachine.lean</code>)	7
4.3 Compiler and wave algebra	8
4.4 The trusted base, per theorem	9
4.5 What ties the model to the code	9
5. Integrity of proof artifacts	10
6. Evaluation	11
6.1 Proof checking	11
6.2 Conformance and differential agreement	12
6.3 Implementations	12
7. Limitations	13
8. Related work	14
9. Conclusion	15
10. Artifact availability	16
Acknowledgements	17
References	17

Abstract

We present Σ -GLYPH Book I, a content-addressed combinator machine in which a single unsigned integer budget — *ATP* — prices both the work an evaluation performs and the *peak* memory it materializes, and we report a layered Lean 4 mechanization of it. The central theorem, `EvalMachine.evalHash_peak_size`, is $size \leq atp + 1$ at **every** configuration the run passes through, proven *on a model of the actual hash-thunk evaluator* rather than on an abstract cost algebra: the per-step accounting lemma `size_step` establishes that every priced action grows the term by at most $cost - 1$, which is exactly the row-by-row correspondence an abstract proof has to assume. A companion theorem, `evalHash_settles`, shows that the machine’s answer is a configuration on which no further action fires — so a run ends because the machine settled, not because a fuel counter cut it off, and every run exits through exactly one of normal form, ATP-exhausted or unresolved-reference. The mechanization is layered down to bytes — a from-scratch FIPS 180-4 SHA-256 in core Lean, serialization injectivity, round-trip canonicity and validation totality — so that redex recognition by hash is *derived* rather than re-axiomatized. Thirty-six theorems across five fronts are guarded; all eleven evaluator theorems have axiom cone `{propext, Classical.choice, Quot.sound}` exactly, with `native_decide` confined to ten theorems in other fronts whose documented trusted base already names the compiler.

We are equally explicit about what is *not* proven. The correspondence between the Lean model and the running implementations is empirical: three differential bridges execute the compiled Lean model against the reference oracle on 33 evaluation vectors, 334 byte buffers and 582 wave cases; two further bridges check a weaker correspondence (861 oracle steps against the premise the proof consumes, and 3000 random closed λ -terms against a transcription of the Lean compiler model); and a three-engine randomized fuzzer generates 5185 evaluation vectors per CI run.

Our second contribution is methodological, and we state it here in one paragraph because it changes how much the first should be believed. The CI machinery that asserts “these theorems hold and rest on these axioms” — not the Lean kernel, the Python around it — was defeated twenty-one times over six internal hardening rounds and five external reviews: by `sorryAx`, by a string literal that blinded the comment stripper, by a `#print axioms` override installed *by the audited module*, by vacuous theorems, by definition gutting that leaves every pinned statement byte-identical, by an audit whose scope came from a config field nothing compiled from, and — six rounds in — by a file walk that was not recursive. Every bypass was reproduced against a green build; not one required a kernel bug. §5 gives three representative vectors and the thesis that unifies them (*a control whose scope is chosen by the thing it controls*); the full taxonomy is a companion paper (Glova 2026b).

1. Introduction

Suppose you are handed a computation by hash and asked to run it. Not a program you wrote; a stranger’s. You want three things before you agree. You want to know it will stop. You want to know how much memory it can make you allocate before it stops. And you want your answer to be the same as everybody else’s answer, bit for bit, so that “I ran it and got X” is a claim someone else can check rather than a claim they must believe.

Existing designs give you subsets. The Ethereum Virtual Machine’s gas (Wood 2014) bounds work and has been mechanized several times (Hirai 2017; Hildenbrandt et al. 2018; Amani et al. 2018), but gas is not a memory theorem: memory is priced through a separate, quadratic schedule, and no statement of the form “peak allocation is a function of the budget” falls out of the semantics. WebAssembly’s deterministic profile (W3C WebAssembly Working Group 2025; Haas et al. 2017) gives reproducible execution and has a mechanized core (Watt 2018), but the memory bound is an ambient property of the linear-memory limit, not a consequence of the budget the caller supplies. Nock (Yarvin et al. 2016) is small and deterministic and has no budget at all. Unison (Chiusano et al. 2024) makes code content-addressed — the identity discipline we also adopt — without a resource semantics. zkVMs (RISC Zero 2025; Succinct Labs 2025; Goldberg et al.

2021) give you a proof that *this* execution happened with *these* cycles, which is a per-execution certificate, not a universally quantified theorem about all inputs.

Σ -GLYPH takes the narrow road. The machine is SKI combinatory logic (Schönfinkel 1924; Curry and Feys 1958) over content-addressed nodes: a term is a directed graph of 34- or 66-byte canonical records, each identified by the SHA-256 of its own bytes, and reduction is leftmost-outermost. There is no clock, no floating point, no network, no ambient state. The whole interface is

$$result_hash = eval(term_hash, atp).$$

The design decision that makes this paper worth writing is the pricing. Under the *hash-leaf size model* (ADR-001 composed with ADR-003), an unresolved hash counts as exactly one node whatever it denotes, materializing one node costs its own materialized size (1, 2 or 3), and the duplicating rule R-S costs $1 + size(z)$ where z is the duplicated argument in its *current* materialization. The consequence, which Book I §3.4 states normatively, is that every action costs strictly more than the size it adds — and therefore along any execution

$$materialized_size(t) - 1 \leq spent.$$

One integer prices both axes. A caller who hands out a budget of n has simultaneously bounded the callee’s work at n actions and its peak materialized graph at $n + 1$ nodes, without a second knob, a second failure mode, or a heap accountant.

Contributions.

1. **A combinator machine whose single budget provably bounds peak memory, with the theorem proven on the concrete evaluator** (§3, §4). `SizeBound.lean` proves the invariant over an abstract seven-row cost algebra with a reachability quantifier; `EvalMachine.lean` re-proves it on a Lean model that mirrors the reference evaluator’s control flow, closing the row-correspondence gap the abstract proof must otherwise assume, and `EvalMachine.evalHash_peak_size` lifts it from the answer to every configuration of the run. The distinction is not cosmetic: on a store-backed *SKKI* the trace peaks at 7 nodes and answers with 1, so a bound on the returned configuration is strictly weaker than a peak bound.
2. **Settling, and outcome classification, as theorems rather than comments** (§4.2). `eval_settles` / `evalHash_settles` establish that `evalHash` returns a configuration on which `step` fires nothing — the machine is settled when it answers — and therefore that every run exits through exactly one of `step`’s three non-firing results, i.e. one of Book I’s three canonical outcomes. Totality and determinism remain *definitional* (a total Lean function with no `partial`, no `unsafe`); what was previously only a source comment is the claim that the fuel counter is never what stopped the machine.
3. **A layered mechanization down to bytes** (§4). `Sha256.lean` is a total, from-scratch FIPS 180-4 SHA-256 in core Lean (National Institute of Standards and Technology 2015); `MachineBytes.lean` proves serialization injectivity, round-trip *and canonicity*, validation totality, and the §4.1 length law; `EvalMachine.lean` is built on top, so recognizing $\langle I \rangle$, $\langle K \rangle$, $\langle S \rangle$ by hash is a derived fact about the proven byte layer, not a fresh axiom.
4. **An honest, per-theorem account of the trusted base** (§4.4, §7). Ten of the 36 guarded theorems are permitted `native_decide`, which puts the Lean compiler in the trusted base; the other 26 are not, and all eleven evaluator theorems — including the peak-memory bound and the settling theorems — depend on the three standard Lean axioms alone. Where a property is empirical, we say so in the sentence that makes the claim.
5. **An account of what the guarded-theorem counts are worth** (§5). The CI apparatus asserting those counts was broken and repaired six times; we give three representative bypasses, the current guarantees, the residual gaps, and the thesis that unifies them — *a control whose scope is chosen by*

the thing it controls. The full twenty-one-vector taxonomy, the six rounds and the external review data are a companion paper (Glova 2026b).

We claim no new proof technique and no new logic. This is a systems and proof-engineering paper: the interest, if there is any, lies in the composition and in the failure modes we document.

2. Background: what “checkable” has to mean

Three properties are load-bearing for the use case — re-executing a stranger’s reason — and they interact.

Identity is bytes. Every node is a `SigmaNodeV2` record with the layout `[Op:1] [Flags:1] [Atom?:32] [Left?:32] [Right?:32]` and `NodeHash = SHA-256(bytes)`. The flags are not free: each opcode has exactly one normative flag value, so there is exactly one byte string per node and the encoding is canonical by construction. This is the same discipline that JCS (Rundgren et al. 2020) imposes on JSON and that multihash (Protocol Labs 2024) and IPFS (Benet 2014) impose on content addressing, and it matters for the same reason: if two byte strings can denote one value, the hash is no longer an identity, and two honest nodes can disagree about what they were asked to compute. Nix (Dolstra 2006) and Unison (Chiusano et al. 2024) make the same bet at different granularities.

Divergence must be a value, not an event. SKI is Turing-complete; $\Omega = (SII)(SII)$ does not terminate. A machine that hangs is not checkable, so the budget is not an optimization but a totality device: when the budget cannot pay for the next action, evaluation returns a `DISSONANCE` node carrying `SHA-256("ATP Exhausted")`, and that node has a hash like any other. The same holds for a demanded hash that is absent from storage (`SHA-256("Unresolved Reference")`) and for bytes that fail validation (`SHA-256("Invalid Object")`). Failure is data.

Memory has to be bounded by the same knob. This is the point where a naive design leaks. Under a flat “one gas per rewrite” schedule, R-S duplicates an arbitrary subterm for constant cost, so term size grows $O(2^{atp})$: a budget of 60 buys gigabytes. ADR-001 records this as an OOM-before-ATP denial of service against validators, and it is not hypothetical — it is the ordinary consequence of pricing rewrites rather than materialization. The literature on resource-aware type systems (Hofmann and Jost 2003; Hoffmann et al. 2012) and on separation logics with time and space credits (Guéneau et al. 2018; Charguéraud and Pottier 2019; Moine et al. 2023) attacks the same problem from the language side, deriving bounds for programs written in a rich language. Our setting is the dual one: the language is fixed and tiny, and the bound is a property of the *machine*, universally quantified over all inputs, so a caller needs no analysis of the term at all.

There is a fourth property, less philosophical: **laziness**. ADR-003 makes materialization demand-driven along the left spine, so a subtree that leftmost-outermost reduction never demands is never fetched. This is what makes withholding data a liveness problem rather than a safety problem, and it composes with the pricing: you pay for exactly what the reduction touches. It also creates the tension that ADR-001 §“Composition” resolves — R-S needs $size(z)$ while ADR-003 exists to avoid touching z — and the resolution is the hash-leaf model: an unresolved hash contributes exactly 1, so measuring z never forces it.

3. The machine

We give the fragment of Book I v0.5.2 that the mechanization covers. The normative source is `spec/book-1-truth.md` (Ukrainian); `spec/book-1-truth.en.md` is an informative translation, and section numbers below refer to both.

3.1 Nodes, bytes, identity

$$Node ::= LITERAL(a) \mid REF(t) \mid APPLY(l, r) \mid DISSONANCE(r)$$

with a, t, l, r, s all 32-byte hashes. Serialization (§2) is

$$\begin{aligned}\text{ser}(\text{LITERAL}(a)) &= 00\ 01\ a \\ \text{ser}(\text{REF}(t)) &= 01\ 01\ t \\ \text{ser}(\text{APPLY}(l, r)) &= 02\ 06\ l \parallel r \\ \text{ser}(\text{DISSONANCE}(r)) &= \text{FF}\ 01\ r\end{aligned}$$

and $\text{NodeHash}(n) = \text{SHA-256}(\text{ser}(n))$. Deserialization (§4.1) accepts a buffer iff the opcode is in the table, the flag byte *equals* the opcode’s normative value, and $\text{len} = 2 + 32 \cdot \text{popcount}(\text{flags})$ — hence 34 or 66 bytes, and nothing else. Opcode 0x03 is permanently reserved-invalid. Any validation failure materializes the *Canonical Invalid Object*, a fixed **DISSONANCE** node; there is no undefined behavior and no exception.

The genesis combinators are nominal, not magic: $\langle I \rangle = \text{LITERAL}(\text{SHA-256}(\text{"I"}))$ and likewise for K and S ; §5.1 pins their NodeHashes. Since v0.5 they are *intrinsic*: an implementation must serve them from the paragraph, not from storage, so a node cannot be denied K . **FALSE** is deliberately not an axiom — it is $\text{APPLY}(\langle K \rangle, \langle I \rangle)$, constructible from the two axiom hashes with no store at all, which the spec phrases as “FALSE is a theorem, not an axiom.”

3.2 Terms, thunks, and the size model

The v0.5 abstract machine (§3.3) operates on *hash thunks*. A term under reduction is a graph of materialized nodes whose children are either materialized nodes or bare 32-byte hashes:

$$T ::= \mathbf{thunk}(h) \mid \mathbf{lit}(a) \mid \mathbf{ref}(t) \mid \mathbf{dis}(r) \mid \mathbf{app}(T, T)$$

Size, the quantity the bound is about, is (§3.4):

$$\text{size}(\mathbf{app}(f, a)) = 1 + \text{size}(f) + \text{size}(a), \quad \text{size}(\mathbf{ref}(\cdot)) = 2, \quad \text{size}(__) = 1$$

A materialized REF counts 2 because it is a node plus the thunk of its target. An unresolved thunk counts exactly 1 *regardless of what it denotes* — this is the hash-leaf model, and it is what lets R-S price its argument without forcing it.

3.3 Reduction and pricing

The rewrite rules (§3.1) are the standard SKI set plus one-level reference unfolding:

$$\begin{aligned}\text{R-I: } & \text{APPLY}(\langle I \rangle, x) \rightarrow x \\ \text{R-K: } & \text{APPLY}(\text{APPLY}(\langle K \rangle, x), y) \rightarrow x \\ \text{R-S: } & \text{APPLY}(\text{APPLY}(\text{APPLY}(\langle S \rangle, x), y), z) \rightarrow \text{APPLY}(\text{APPLY}(x, z), \text{APPLY}(y, z)) \\ \text{R-R: } & \text{REF}(h) \rightarrow \text{resolve}(h) \quad (\text{exactly one level per step})\end{aligned}$$

Reduction is leftmost-outermost, i.e. normal order (Barendregt 1984), so a redex is reduced before its arguments. Combinator recognition is *identity by hash* (§3.2): a node is $I/K/S$ iff its NodeHash equals the §5.1 constant. Crucially this test applies to thunks without materializing them, which is what makes redex recognition $O(1)$ and lazy simultaneously.

Prices (§3.4):

$$\begin{aligned}\text{cost}(\mathbf{force}\ h) &= \text{size of the materialized node} = 1 \mid 2 \mid 3 \\ \text{cost}(\text{R-R}) &= \text{cost}(\text{R-I}) = \text{cost}(\text{R-K}) = 1 \\ \text{cost}(\text{R-S}) &= 1 + \text{size}(z)\end{aligned}$$

Two disciplines make this total and deterministic. First, **the exhaustion check precedes the action**: an action whose price exceeds $atp - spent$ is not performed, and evaluation returns ATP Exhausted with **spent** unchanged. In particular $\text{eval}(\text{REF}(\text{missing}), 0)$ is ATP Exhausted, decided before any storage access — the outcome cannot depend on what the store contains. Second, the minimum price of any action is 1, so evaluation strictly draws down the budget and cannot spin at zero cost.

3.4 The bound

Every rule’s size increment is strictly less than its price. R-I and R-K shrink; force-LITERAL is size-neutral at cost 1; force-REF adds 1 at cost 2; force-APPLY adds 2 at cost 3; R-R turns a 2-node REF into a 1-node thunk at cost 1; and R-S, the only growing rule, adds $\text{size}(z) - 1$ at cost $1 + \text{size}(z)$ — it duplicates z but discards the $\langle S \rangle$ spine node, and under the hash-leaf model copying an unresolved leaf adds exactly one node. Hence along any execution

$$\text{size}(t) - 1 \leq \text{spent} \leq \text{atp},$$

at every point of the execution and not merely at its end, which gives a caller a preflight memory estimate of $\text{atp} + 1$ nodes. Book I §3.4 adds a warning that a 2026-07 review forced: because **spent** is an *upper*, not a lower, bound on size, an implementation must not use **spent** as a live fault trigger — Ω is a tiny term that spends everything — and a guard must measure actual **size(t)**.

3.5 Canonical compilation

Book I §6 fixes exactly one $\lambda \rightarrow \text{SKI}$ compiler, Profile C1, so that two humans writing the same λ -term get the same hash. It is textbook bracket abstraction (Turner 1979) with the rules applied *strictly in order*: $A(x, x) = \langle I \rangle$; then $A(x, M) = K M$ when $x \notin FV(M)$; then $A(x, MN) = S A(x, M) A(x, N)$. The ordering is normative and is not an optimization: C1 performs no η -reduction and does not canonicalize extensionally, so $C1[\lambda x. \lambda y. x] = S(KK)I \neq \langle K \rangle$ — a separate citizen, extensionally equal to K . Rice’s theorem is the reason the spec gives, and it is the right one: syntactic canonicity is achievable, semantic canonicity is not.

4. The mechanization

All proofs are Lean 4 v4.31.0 (Moura and Ullrich 2021), **core only, no mathlib**. The pinned toolchain is `proofs/lean-toolchain`. Ten `.lean` files total 1304 lines: `Sha256.lean` (110), `MachineBytes.lean` (282), `EvalMachine.lean` (340), `WaveAlgebra.lean` (203), `C1Compiler.lean` (138), `SizeBound.lean` (95), `LutData.lean` (17 lines holding a 206 KB generated table), and three `*Run.lean` I/O runners (49 + 49 + 19) which prove nothing and exist to feed the differential bridges.

The five fronts are *layered, not independent*: `Sha256` $\rightarrow$ `MachineBytes` $\rightarrow$ `EvalMachine` is a genuine dependency chain, so the evaluator’s redex recognition is stated over the proven serialization layer rather than over an axiomatized `is_glyph` predicate. This is the structural difference from a mechanization that models reduction and treats byte identity as given.

4.1 Bytes (`Sha256.lean`, `MachineBytes.lean`)

`Sha256.sha256` : `List UInt8` $\rightarrow$ `List UInt8` is FIPS 180-4 written out: the round constants, `H0`, the four Σ/σ functions, `pad` (whose zero count is computed arithmetically so the definition is total by construction), and a compression loop over `Std.Range`. There is no `partial`, no `unsafe`, no `extern`. What is *not* claimed: there is no proof that this function is SHA-256. Its correctness is established differentially, against FIPS digest vectors and against the oracle, and we say so in §7.

On top of it, `MachineBytes.lean` proves:

Theorem	Claim
<code>serialize_injective</code>	distinct well-formed nodes never share canonical bytes
<code>deser_serialize</code> <code>serialize_deser</code>	round-trip: parsing canonical bytes returns the node canonicity : a valid buffer <i>is</i> the serialization of its parse — no second byte form exists for any node
<code>deser_wf</code>	validation only ever yields well-formed nodes (totality of the parser)
<code>valid_lengths</code>	§4.1 rule 3: a valid buffer is 34 or 66 bytes
<code>reserved_opcode_invalid</code>	§1.2: 0x03 never parses
<code>lit_bytes_disjoint</code>	byte-0 discrimination under <code>glyph_eq</code> ’s $O(1)$ redex test

`serialize_deser` is the one that earns its place. Round-trip alone ($\text{deser} \circ \text{ser} = \text{id}$) is compatible with a second, non-canonical encoding of the same node; canonicity ($\text{ser} \circ \text{deser} = \text{id}$ on valid buffers) is what makes the hash an identity. Together with `serialize_injective` it says the map from well-formed nodes to valid buffers is a bijection, and the hash layer above adds only the collision assumption, which is stated and not proven.

Five further theorems pin the spec’s constants end-to-end: `genesis_I`, `genesis_K`, `genesis_S` recompute $\text{SHA-256}(\text{ser}(\text{LITERAL}(\text{SHA-256}(\text{"I"}))))$ inside the proof assistant and compare against the §5.1 hex; `false_is_a_theorem` does the same for $\text{APPLY}(\langle K \rangle, \langle I \rangle)$; `invalid_object_pins` for the §4.2 object. These five use `native_decide` and are the byte front’s only entries in the compiler-trusting set.

4.2 The evaluator (`EvalMachine.lean`)

`EvalMachine.step` : $\text{Term} \rightarrow \text{Nat} \rightarrow \text{Store} \rightarrow \text{StepResult}$ mirrors the reference implementation’s `step5`: leftmost-outermost descent along the left spine, redex heads recognized by `glyphEq` (a hash comparison that is `false` on any application, so R-K and R-S fire only on the exact spine shapes), thunk forcing priced by the materialized node’s size, and the genesis intrinsic handled before any store lookup. `eval` iterates it under fuel; `evalHash h atp st` starts from `.thunk h` with fuel `atp + 1`.

Totality and determinism are definitional, not theorems. `step` is well-founded on `sizeof t`; `eval` is structural recursion on fuel; `evalHash` is a total Lean function with no `partial` and no `unsafe`. Determinism is the statement that it is a function. This is a stronger form of the property than a theorem about a relation, and it is also a weaker claim than it may sound: it is a fact about the Lean model, and the model’s agreement with any implementation is empirical (§4.5, §6). Definitional totality also says less than one wants: it guarantees `evalHash answers`, not that the answer is a settled configuration rather than a trace truncated by the fuel counter. That gap is what `eval_settles` closes.

The proven statements:

- `step_bounds` — for any fired action, $1 \leq c \leq \text{rem}$. Proved by `fun_induction step` followed by `grind [size_pos, size]`. The file documents the manual skeleton `grind` automates, per branch, because a tactic that discharges eight cases at once is exactly where a reader loses the ability to audit. This transparency note was added in response to a 2026-07 focused review.
- `step_cost_le`, `step_cost_pos` — its two projections: an action never costs more than the remaining budget, and never costs zero.
- `eval_spent_le`, `evalHash_spent_le` — $\text{spent} \leq \text{atp}$ for **all** terms, budgets and fuel. Previously a per-vector observation; here a theorem.
- `eval_settles`, `evalHash_settles` — **fuel sufficiency**: whenever $\text{atp} - \text{spent} < \text{fuel}$, `step (eval fuel t atp spent st).1 (atp - (eval ...).2) st = .nf`. The proof is a four-case induction: each accepted action costs at least 1 (`step_cost_pos`) and at most what remains (`step_cost_le`), so the remaining budget strictly decreases and the fuel counter cannot be what stopped the machine. `evalHash_settles` instantiates it at the fuel `evalHash` actually passes, `atp + 1`.

- **size_step** — the exact §3.4 per-step accounting: $size(t') + 1 \leq size(t) + c$. R-S is the only growing rule and holds *unconditionally*: the discarded $\langle S \rangle$ head is pure slack, so no leaf assumption on z is needed.
- **eval_size_bound**, **evalHash_size_bound** — the ADR-001 bound, $size(result) \leq spent + 1$, on the concrete evaluator.
- **evalHash_peak_size** — **peak** memory: $size(eval\ k\ (.think\ h)\ atp\ 0\ st).1 \leq atp + 1$ for every k . The quantifier over k is what makes it a peak bound: **eval k** runs at most k actions and its fuel-out branch returns the configuration reached, so $k = 0, 1, 2, \dots$ enumerates the configurations of the $atp + 1$ run. The proof composes **eval_size_bound** at each k with **eval_spent_le** to replace $spent_k$ by atp .

Outcome classification, stated at the right scope. **evalHash_settles** says **step** fires nothing on the returned configuration; **eval** reaches its exits only through **step**’s **.nf**, **.exhausted** and **.unresolved** results, whose returned terms are respectively the settled term, **.dis rATP** and **.dis rUnres**. So every run exits through **exactly one of three** — normal form, **DISSONANCE(ATP Exhausted)**, **DISSONANCE(Unresolved Reference)**. What is *not* claimed, and the artifact is explicit about this, is that the three outcome *terms* are pairwise distinct: a term materialized from the store can itself reduce to the ATP-Exhausted leaf, so a normal-form exit may return the same term an exhausted exit would. The trichotomy is a property of the machine’s **exit**, not of the term it hands back. A reader who wants to distinguish the three from the result hash alone cannot; that is a deliberate consequence of failure being ordinary data.

The two theorems are also non-vacuous in the way one should check. For a store-backed *SKKI* at $atp = 50$, the $(k, size, spent)$ trace is $(0, 1, 0), (1, 3, 3), (2, 5, 6), (3, 7, 9), (4, 7, 11), (5, 1, 12)$ — peak 7, returned configuration 1 — so **evalHash_size_bound** genuinely does not imply **evalHash_peak_size**, and the fuel quantifier is load-bearing. On the same trace **step** fires on the returned configuration for $k = 0..4$ and is **.nf** only from $k \geq 5$, so **eval_settles**’s hypothesis $atp - spent < fuel$ is load-bearing too. (We reproduced both traces against the branch’s unmodified **EvalMachine.lean**.)

Status. All eleven evaluator theorems are guarded on **master**. The three that close the settling and peak claims — **eval_settles**, **evalHash_settles**, **evalHash_peak_size** — were developed on a branch and merged at **35d8aea**, where they were re-pinned against a guard that had moved twice underneath them (§5.4, §5.5). We verified the merged state independently: the eval bridge reports eleven guarded theorems with standard axioms only, and we reproduced both traces above against **master**’s unmodified **EvalMachine.lean**. One nuance the project’s own governance discipline demands: the most recent *adopted* anchor set is **v0.6.7** at **16a1355**, and these three theorems merged after it, so they are on the branch in force but are not covered by an adoption warrant (§7).

This is the paper’s main technical claim, so it is worth being precise about what **SizeBound.lean** adds and what it does not. **SizeBound.lean** abstracts the machine to a pair $\langle size, spent \rangle$, gives seven constructors for the priced actions, and proves **memory_bound** over an inductive reachability relation — so it quantifies over *all reachable states*, which is the peak-memory statement. But the seven rows are *assumed* to correspond to the real machine’s actions; the file says so in its header. **EvalMachine.size_step** discharges that assumption, on the real control flow, with no classifier, and **evalHash_peak_size** supplies the missing quantifier. The two files are now complementary in a narrower sense than before: **SizeBound** has a reachability relation over an abstract model, while **EvalMachine** has both the correspondence and a peak bound over the concrete one. We keep **SizeBound** in the guarded set because its seven-row presentation is what Book I §3.4 normatively states and what an independent implementer would check against — not because the evaluator proof depends on it, which it does not.

4.3 Compiler and wave algebra

C1Compiler.lean models §6 and proves **mem_skiFv_abstr** ($A(x, \cdot)$ removes exactly x from the free variables), **mem_skiFv_c1** (C1 preserves free variables exactly) and **c1_closed** (a closed λ -term compiles to a variable-free SKI term). The last one converts a runtime check into a theorem: the reference implementation’s “free variable escapes abstraction” guard *cannot fire* on closed input. Two TV-10 vectors are pinned by **rf1**. This front’s entire axiom cone is **{propext}** — the guard’s allowed set for it is exactly that, not the broader standard set, so the documented claim and the enforced claim are the same sentence.

`WaveAlgebra.lean` mechanizes Book II’s integer interference algebra, which sits *above* identity as an optional navigation view and never inside it. It proves range closure (`interfere_valid`), the zero-amplitude cascade, the Law of Left Dominance (`left_dominance_ph`), the Resonance Identity (`crystallization`: the unique non-zero fixed point of self-interference is $\{am = 65535, en = -32768\}$), and two negative results used as design evidence — `fold_not_associative` and `not_commutative`, which are the machine-checked form of the ADR-006 argument that folding interference is unsound. We include this front because it is part of the artifact, not because Book I depends on it; ignoring Book II entirely leaves Book I intact.

4.4 The trusted base, per theorem

Thirty-six theorems are guarded, distributed as: `size` 2, `bytes` 12, `eval` 11, `wave` 6, `c1` 5. The registry pins **39** statements: the three extra are unguarded wave lemmas whose `native_decide` trust axioms guarded theorems borrow, pinned in what they say — see (Glova 2026b). Seventeen further theorems are registered as deliberately unguarded (helper lemmas: list-append inverses, LUT range facts, `divRoundHalfUp` bounds). The per-front axiom policy is machine-enforced, not documentary:

Front	Allowed axioms	<code>native_decide</code> permitted for
<code>size</code>	<code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code>	— (none)
<code>eval</code>	<code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code>	— (none)
<code>bytes</code>	<code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code>	the 5 genesis / FALSE / invalid-object pins
<code>wave</code>	<code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code>	5 of 6 (<code>left_dominance_ph</code> excepted)
<code>c1</code>	<code>propext</code> only	— (none)

So `native_decide` — which evaluates a decision procedure with the compiled evaluator and therefore adds the Lean compiler to the trusted base, and which `mathlib` disallows for exactly this reason (The `mathlib` Community 2020) — is confined to ten of the 36 guarded theorems, each of which is a *concrete computation* (a digest, a 32769-entry LUT scan, a 65536-case amplitude fixed-point scan) rather than a structural claim. In particular the evaluator front, which carries every claim this paper leads with, admits none: we verified independently, with `#print axioms` at the toolchain the repository pins, that `evalHash_peak_size`, `evalHash_size_bound`, `eval_settles`, `evalHash_settles` and `evalHash_spent_le` each depend on `{propext, Classical.choice, Quot.sound}` and nothing else. The wave front’s honesty note is sharper than the headline: `interfere_valid` is *reasoning*-symbolic but consumes `lut_range`, which is `native_decide`, so its soundness rests transitively on the compiler too. Only the pure integer lemmas and `left_dominance_ph` are compiler-independent in that front.

4.5 What ties the model to the code

Nothing in Lean does. The Lean model is a Lean model. What ties it to the running system is five differential bridges, in the tradition of differential testing (McKeeman 1998; Yang et al. 2011):

- `eval_bridge_check.py` executes the compiled Lean evaluator via `EvalRun.lean` and compares result `NodeHash` and `atp_spent` against the reference oracle on all 33 evaluation vectors.
- `byte_bridge_check.py` runs the Lean pipeline on 334 buffers — every conformance CAS object including the deliberately malformed Era-1 0x03 one, the genesis bytes, and roughly 250 adversarial mutations (truncation, out-of-mask flags, wrong-in-mask flags, reserved opcode, op/flag swap) — comparing CAS keys, §4.1 verdicts and round-trips.
- `wave_bridge_check.py` regenerates `LutData.lean` byte-identically and compares the Lean `interfere` against the oracle on a 582-case deterministic boundary grid.
- `c1_bridge_check.py` compares a Python transcription of the Lean `abstr/c1` against the oracle on 3000 random closed λ -terms, `NodeHash`-exact.

- `bridge_check.py` drives the oracle step-by-step over adversarial terms and asserts the `SizeBound` premise ($\Delta \text{size} \leq \text{cost} - 1$) on every observed action.

Two of these five deserve a warning label, and the repository supplies it: only three (`byte`, `eval`, `wave`) actually execute compiled Lean. `bridge_check.py` checks the Python oracle against the *premise* the Lean proof consumes, and `c1_bridge_check.py` compares a hand-written transcription. And no differential can ever exercise a `Prop`-valued definition — `Valid`, `Wf`, `Reach`, `Step`, `Inv` — which is precisely where each theorem’s hypotheses live. That gap is what §5 is about.

5. Integrity of proof artifacts

A machine-checked theorem in a repository makes two claims. The first is mathematical: the kernel accepted a proof term. The second is bureaucratic: *the thing the kernel accepted is the thing the README says it accepted*. The first claim is what proof assistants are for and it is in excellent shape. The second is a software supply-chain problem (Torres-Arias et al. 2019; OpenSSF 2025), it is the one a reader of §4 actually depends on, and in our experience it is where everything went wrong.

This section states the problem and our position on it. The full account — a taxonomy of twenty-one reproduced bypasses over six internal hardening rounds and five external reviews — is a companion paper (Glova 2026b); we summarize only what a reader of the mechanization needs in order to calibrate how much the guarded-theorem counts in §4.4 are worth.

The starting point. `lean` exits 0 on a file containing `sorry`. It is a warning. So “CI compiled the proofs” is not “CI checked the proofs”, and a repository that gates on compilation gates on nothing in particular. The standard remedy is a script that compiles, queries each headline theorem’s axiom dependencies with `#print axioms`, and fails the build on anything unexpected. Ours was about forty lines. Over eight days of adversarial review it became 1465 lines of Python, a 175 KB pin registry and a 981-line regression suite asserting 122 properties — machinery $2.3\times$ the size of the 1304 lines of Lean it guards. Every bypass in that history was demonstrated end-to-end: not “this check has a theoretical gap” but *here is a file, here is the green CI output, here is the falsehood it certified*.

Three representative vectors, chosen because each defeats a defense a reader would reasonably assume sufficient.

An axiom check verifies how something was proven, never what. Replacing a theorem’s statement with `theorem memory_bound : True := trivial` leaves the axiom cone clean — `trivial` needs no axioms — so the guard printed “OK `#print axioms` clean (std axioms only)” and the bridge printed `PREMISE HOLDS ON ALL OBSERVED STEPS`, exit 0. The same hole admits a weakened hypothesis or an altered conclusion. The fix is to pin each guarded theorem’s *elaborated statement*, structurally dumped so notation cannot make the displayed form differ from the elaborated one.

Pinning a statement does not pin what the statement is about. Deleting one constructor from the inductive reachability relation of `SizeBound.lean` — a four-line diff — shrinks `memory_bound` to a claim about the single state $\langle 1, 0 \rangle$; `def Valid (_w : Wave) : Prop := False` makes `interfere_valid`, `left_dominance_ph` and `crystallization` simultaneously vacuous. In every case every pinned statement dump stayed byte-identical and every bridge stayed green, including `WAVE-BRIDGE: ALL AGREE (582/582)`. The fix is to pin, from the kernel environment, the transitive definition dependency set of each guarded statement — a definition by type *and* value, an inductive by type *and* constructor list — and to treat an unpinned dependency as a hard failure so a new one cannot appear silently.

An audit whose scope is a configuration field is not an audit. The definition-pinning scope was read from a per-front `build` list that nothing actually compiled from, and the driver silently dropped every constant owned by a module outside it. Deleting one module name therefore disabled the whole control for that front and restored both of the PIs above: `wave build` shortened by one entry, plus `Valid := False`, gave `WAVE-BRIDGE: ALL AGREE (582/582)`, rc 0, *while printing* “every definition they are stated in terms of

matches its pin”. The fix is an inversion — derive the scope as the complement of a small, separately declared core-Lean allowance, so it cannot shrink by editing a list the same commit edits.

The thesis. These are not unrelated bugs. Every vector we found instantiates one shape: **a control whose scope is chosen by the thing it controls**. The axiom cone is chosen by the theorem’s own proof; the axiom *answer* was, until we fixed it, chosen by the audited module, which could install syntax overriding `#print axioms` and hand the guard a fabricated list; the comment stripper’s view of a file was chosen by a string literal inside it; the set of declarations by their own line breaks; the set of files by which directory they sit in. The defense that generalizes is not a longer denylist but that inversion: derive scope from something the audited artifact cannot edit, and make an unpinned, unregistered or unclaimed entity a failure rather than a silent skip. Two corollaries recur — *UNRUN is not PASS*, and *a scan that finds nothing is indistinguishable from a scan that never ran*.

What this means for §4. Not one of the twenty-one bypasses required a bug in the Lean kernel; every one type-checked. Independent re-checkers such as `lean4checker` (Lean FRO 2024) and `lean4lean` (Carneiro 2024) raise assurance about the mathematical claim and would have caught none of them, because the proofs really are valid — of the wrong proposition. So the counts in §4.4 rest on machinery that has been broken six times and repaired six times, most recently by an outside reviewer who observed that the guard’s file walk was not recursive, so a `proofs/Sub/Evil.lean` containing `axiom backdoor : False` was read by nothing while CI exited 0. We state the current guarantees precisely — statements and transitive definition dependencies pinned from the kernel environment, axioms queried through `Environment.importModules` as data rather than elaboration, scope derived rather than configured, the guarded set claimed by identity, and one recursive enumeration feeding every source-layer check — and we note the residual gaps rather than footnote them: no differential can exercise a `Prop`-valued definition, which is exactly where each theorem’s hypotheses live; the `*Run.lean` runners are unproven I/O plumbing and the one place `partial` is allowed; regeneration can launder any drift and is therefore never run by CI; and `GUARD_CLAIMS.txt` is a review-visibility control, not an authority. The reader who wants the evidence behind those sentences should read the companion paper; the reader who wants only the calibration should take away that §4’s numbers are as good as a human reading of one JSON diff. —

6. Evaluation

All figures below were measured at commit `35d8aea` on `master`, on an Apple-silicon macOS host (arm64-darwin 24.6.0) with Lean 4.31.0, from a clean tree extracted with `git archive` and containing no `.olean` cache. Every number is reproducible with the commands named.

6.1 Proof checking

Bridge	Wall (s)	Guarded theorems	Differential
<code>bridge_check.py</code> (SizeBound)	1.10	2	861 oracle steps, 0 premise violations
<code>byte_bridge_check.py</code>	1.96	12	334 / 334 buffers agree
<code>eval_bridge_check.py</code>	4.30	11	33 / 33 eval vectors agree (hash and atp_spent)
<code>wave_bridge_check.py</code>	33.14	6	582 / 582 boundary cases agree
<code>c1_bridge_check.py</code>	1.19	5	3000 random closed λ -terms agree
all five, one cold sequential run	42.3	36	
<code>tests/proof_guard_test.py</code>	38.86	—	122 checks, 0 failures

The per-bridge figures and the all-five figure are separate measurements (the per-bridge column sums to 41.7 s). Each bridge builds its own `.olean` artifacts in a private temporary directory, so no figure benefits from a warm cache. The guard regression suite now costs about as much as all five bridges together, because most of its 122 checks compile a fixture through `lean`.

Adding the three theorems of §4.2 did not measurably change the evaluator bridge’s cost — they are short inductions over already-elaborated definitions and add no new pinned definitions, the dependency set staying at 155 constants. The equational form of `eval_settles — step ... = .nf` rather than a `match ... | .step _ _ => False` phrasing — was chosen for that reason: the registry pins compiler-generated auxiliaries like `EvalMachine.step.match_3` alongside ordinary definitions, and a match-shaped statement would have added another.

The wave front dominates the bridges: `LutData.lean`’s 32769-entry table and the `native_decide` scans account for roughly four-fifths of the total. Bare kernel type-checking of the small files is negligible — `SizeBound.lean` 0.20 s, `C1Compiler.lean` 0.27 s, `Sha256.lean` 0.21 s — so the wall time is guard machinery and compiled evaluation, not proof search.

6.2 Conformance and differential agreement

`tests/spec_conformance/vectors.json` holds **49** vectors: 33 `eval`, 8 `object` (serialization/hash), 8 `deserialize` (byte-rejection), over 36 preloaded CAS objects one of which is deliberately malformed. The evaluation vectors’ outcome distribution is 17 normal form, 12 ATP-exhausted, 3 unresolved-reference, 1 invalid-object — i.e. more than half exercise a failure path, which is the point: in this design the failure paths are the consensus-critical ones.

The property suite (`tests/spec_conformance/test_properties.py`, fixed seed 0x516) runs **2103 checks in 0.04 s**, of which P7 — the memory-bound property — is **150 random traces**, and P2 is deserialization totality over **2000 random byte buffers**. (We flag this because `SizeBound.lean`’s header comment described P7 as covering “~2000 random traces” until commit `a58d5d6` corrected it: the 2000 belongs to P2 and the 2103 is the suite-wide check count. The same commit corrected the conformance README’s governance-scenario count from 16 to 20. Both are small, and both are the kind of drift §5 is about.)

The randomized three-engine fuzzer `tests/book1_fuzz.py` generates random SKI terms (depth 1–5, mixing genesis leaves, random literals, resolvable REFs and deliberately absent “ghost” REFs) crossed with an ATP grid per term — including 0, *spent* − 1, *spent*, *spent* + 1 and $2^{32} - 1$ — and replays every generated vector through three engines: the Python oracle, the Rust implementation, and `warrant-go`’s native evaluator. CI runs three fixed seeds at 200 terms each on every push, which we measured as 1706, 1737 and 1742 vectors — **5185 generated evaluation vectors per CI run**, all agreeing, in about 0.14 s per seed. The seeds are fixed so that a divergence is reproducible.

6.3 Implementations

Implementation	Lines	SHA-256	Book I evaluator
<code>impl/sigma_glyph.py</code> (oracle)	448	<code>hashlib</code>	yes
<code>impl-rs/src/main.rs</code>	778	from scratch, zero crate dependencies	yes
<code>warrant-go</code> (external repo, CI-pinned)	—	Go stdlib	yes
<code>impl-go/main.go</code> (in-tree)	1737	Go stdlib	no — Book III / governance only

The Rust implementation is worth a sentence: 778 lines, no dependencies at all, its own SHA-256, `overflow-checks = true` in the release profile, and it replays the oracle-generated vectors byte-exact. The

in-tree Go implementation covers Books II–III and governance and contains no Book I evaluator; the third Book I engine is `warrant-go`, in a separate repository, pinned in CI by commit hash. Anyone reading “three independent implementations” should read it that way.

The Lean artifact is 1304 lines across ten files. The integrity machinery around it — `proof_guard.py` (1465), `theorem_pins.json` (175 KB), `tests/proof_guard_test.py` (981) and five bridge scripts (731 lines total) — is now **2.3×** the size of the proofs it guards, up from 1.8× before the last three rounds. That ratio is the honest cost of the second claim in §5. It is also the strongest quantitative argument for §7’s concern that the guard work, while the most visible output of this period, may not have been the most valuable one.

7. Limitations

We list these in descending order of how much they should change a reader’s confidence.

The guard work may be the streetlight, not the keys. The one external reviewer that executed the suites found no bypass and instead attacked the priority: hardening a proof guard is defence against adversarial third-party proofs, and this project has had zero external contributors, so six rounds went to hypothetical pull requests from oneself while the friction a first real user meets — policies authored by hand in SKI combinators — went unaddressed. We think that is largely correct, we record it because suppressing it would undercut the paper’s own method, and we do not resolve the tension in our favour; the companion paper argues both sides at length (Glova 2026b). For §4’s purposes the consequence is narrow: the guarded-theorem counts rest on machinery whose development was chosen for legibility rather than by a threat model.

One implementation lineage. The Python oracle, the Rust implementation and `warrant-go` are all written by the same author (with model assistance) from the same specification text, in a period of weeks. Agreement between them is real evidence about specification ambiguity and about coding slips, and it is weak evidence about *specification* error: three implementations of a wrong sentence agree perfectly. Nor is the specification yet in a state that makes an independent implementation easy: the normative Book I is Ukrainian, the English rendering is explicitly informative, and §5.1 defers the three 32-byte genesis atom values to `impl/sigma_glyph.py` rather than printing them — so an implementer must currently read the reference code. No external party has implemented Book I. That is the single most valuable missing datum in this paper.

The model-code gap is empirical and finite. No theorem relates `EvalMachine.lean` to `impl/sigma_glyph.py`. What relates them is 33 evaluation vectors, 334 byte buffers, 582 wave cases, 3000 λ -terms and 5185 fuzzer-generated vectors per CI run — a large finite sample of a space we make no claim to have covered. There is no verified refinement, no extraction, and no proof that the Lean `Store` (a list with linear `find?`) is a faithful model of a content-addressed store.

The trichotomy is about the exit, not the term. `evalHash_settles` proves every run ends on a settled configuration and therefore exits through exactly one of `step`’s three non-firing results. It does **not** prove the three outcome *terms* are pairwise distinct, and they are not: a term materialized from the store can itself reduce to the ATP-Exhausted leaf, so a normal-form exit can return the term an exhausted exit would have returned. A caller who needs to distinguish “it finished” from “it ran out” must be told the exit, not handed the result hash. Relatedly, nothing in the evaluator front proves that the `.dis rATP / .dis rUnres` leaves carry the reason strings Book I §5.3 mandates; that correspondence lives in the byte front and in the vectors’ `result_hash`.

native_decide is in the trusted base for ten theorems. Every genesis hash pin, the `FALSE` constant, the Canonical Invalid Object, and five of six wave theorems trust the Lean compiler in addition to the kernel. The evaluator front does not. We consider the byte-front pins acceptable because they now carry an independent differential against the oracle, and we consider the wave front’s dependence properly disclosed rather than eliminated.

SHA-256 is not proven to be SHA-256. `Sha256.lean` is a transcription of FIPS 180-4 (National Institute of Standards and Technology 2015), checked against FIPS digest vectors and the oracle. There is no equivalence proof against a reference specification of the kind HACL* provides (Zinzindohoué et al. 2017). Collision resistance is assumed, not proven, and the specification says so.

The reported commit is ahead of the adopted one. Everything this paper describes is on `master` at `35d8aea`. The most recent governance-adopted anchor set is `v0.6.7` at `16a1355`; the three evaluator theorems of §4.2 merged after it. So they are on the branch in force but are not covered by an adoption warrant, and under this project’s own rules location is a fact about git while adoption is a threshold signature. A reader who wants only adopted material should check out the tag and will find eight evaluator theorems rather than eleven.

The threshold was met; independent custody was not. The `v0.6.7` adoption warrant carries two valid signatures against a 2-of-3 policy — but the signers are the human principal (`s0fractal`) and a *delegated model actor* (`claude-fable-5`) operating under that human’s authority, not two independent parties. The project documents this itself and counts it: of the six times the threshold has been exercised, three pair two keys that sat in one directory on one host, which is one custody and not a quorum. The maintainer offered a third roster key and the signing actor declined to use it, on the grounds that a cryptographically valid signature attributing its act to a different model would be exactly the provenance forgery the project exists to prevent. We report the adoption as a satisfied *policy*, never as independent attestation, and we note that the project’s own security-assumptions document says no independent gate has run against any of this.

No review of this work was independent. All six internal guard rounds were fresh-context sessions with models from a single family, prompted by the author. Five external reviews crossed model families but not the prompting operator or the framing, which is the definition the project itself uses when it declines to call them a gate: no independent adversarial gate has run against any of this, and none of the guard commits carries the project’s own governance adoption. One external reviewer’s finding was worth all six internal rounds on the dimension that mattered — it saw the assumption the internal attacks shared — and four reviewers between them produced four confident P0 claims that a single command refutes. Both halves of that sentence should be weighted; §5 of the companion paper gives the full table (Glova 2026b).

Reproduction is not read-only. `tests/proof_guard_test.py` writes and then deletes a real `proofs/Helper.lean` while testing the unaudited-file vector. Run it on a clean tree.

Finally, an honest note about scope: this paper describes Book I plus the Book II algebra. Books II and III (navigation and federation) exist, are specified, and are mechanized only in the narrow sense reported in §4.3. Nothing about federation is proven here.

8. Related work

Verified systems. `seL4` (Klein et al. 2009), `CompCert` (Leroy 2009) and `CakeML` (Kumar et al. 2014) establish the discipline this work aspires to and exceed it by a wide margin on the axis that matters most: they prove refinement between a specification and executable code. We do not. Our contribution on that axis is negative and methodological — we report precisely which link is missing and what empirical evidence stands in for it.

Resource bounds. Automatic amortized resource analysis (Hofmann and Jost 2003; Hoffmann et al. 2012) and separation logics with time credits (Guéneau et al. 2018; Charguéraud and Pottier 2019) derive per-program bounds in rich languages; Moine et al. (Moine et al. 2023) address heap space specifically, which is the closest work in spirit to our memory theorem. The difference is quantification: they bound a given program, we bound a fixed machine over all programs, which is possible only because the machine is small enough to make the per-rule inequality $\Delta_{size} < \text{cost}$ hold uniformly.

Metered virtual machines. The EVM (Wood 2014) has been mechanized in Isabelle/HOL (Hirai 2017; Amani et al. 2018) and in K (Hildenbrandt et al. 2018); these are far larger and more mature mechanizations

than ours, of a far larger machine. To our knowledge none of them yields a theorem of the form “peak memory $\leq f(\text{gas})$ ”, because the EVM’s memory expansion is a separate priced resource rather than a consequence of the instruction schedule. WebAssembly (Haas et al. 2017; W3C WebAssembly Working Group 2025) with a mechanized core semantics (Watt 2018) and a deterministic profile gives reproducibility but places memory bounds in the embedder. Nock (Yarvin et al. 2016) is comparable in size and minimality and has no budget.

Content-addressed computation. Unison (Chiusano et al. 2024) is the closest sibling: code is identified by the hash of its normalized form, which is the same “hash-is-identity” bet. Nix (Dolstra 2006), IPFS (Benet 2014) and multiformats (Protocol Labs 2024) apply it to artifacts rather than to reduction. None of these prices execution.

Verified execution certificates. zkVMs (RISC Zero 2025; Succinct Labs 2025; Goldberg et al. 2021) give a succinct proof that a specific execution occurred within specific limits. That is strictly stronger *per execution* and strictly weaker as a design property: it certifies a run, not a machine. The two are complementary; a size-priced machine is a plausible zkVM guest precisely because its resource behavior is a theorem.

Canonicalization and signature pitfalls. Our byte-canonicity theorems (`serialize_deser`, `serialize_injective`) are the combinator-machine analogue of JCS (Rundgren et al. 2020), and the failure mode they exclude is the one Chalkias et al. (Chalkias et al. 2020) document for EdDSA (Josefsson and Liusvaara 2017): an underspecified encoding admits multiple valid representations, and independent implementations then disagree about identity. That the sibling Warrant protocol (Glova 2026c) signs Σ -GLYPH-addressed records is why we treat this as a security property rather than an aesthetic one.

Proof-artifact integrity. The adjacent literature concerns the trustworthiness of the checker itself — the de Bruijn criterion and Pollack-consistency (Barendregt and Wiedijk 2005; Wiedijk 2012), Lean’s type theory (Carneiro 2019), independent re-checkers (Lean FRO 2024; Carneiro 2024) — and proof engineering at scale (Ringer et al. 2019); the supply-chain framings of in-toto (Torres-Arias et al. 2019) and SLSA (OpenSSF 2025) treat build provenance rather than the semantics of what was built. We are not aware of prior work cataloguing attacks on the CI machinery that asserts a repository’s claims about its own proofs, which is the gap the companion paper (Glova 2026b) addresses and which §5 summarizes.

9. Conclusion

The technical result is small and, we think, clean: if you price materialization by materialized size and price duplication by the current size of what is duplicated — counting an unresolved hash as exactly one node — then every action of a leftmost-outermost combinator machine costs strictly more than the size it adds, and one integer budget bounds both work and peak memory. The proof is mechanized twice: once over an abstract cost algebra with a reachability quantifier, and once on a Lean model of the concrete hash-thunk evaluator, whose per-step accounting lemma discharges the correspondence the abstract version has to assume. The mechanization stands on a proven byte layer down to a from-scratch SHA-256, so redex recognition by hash is derived rather than assumed.

The result we did not expect to be writing about is the second one. The CI apparatus asserting that these theorems hold, and rest on these axioms, was defeated twenty-one ways without a single kernel bug — and, after six adversarial rounds against it, by the plain fact that its file walk never entered a subdirectory. Each bypass was a green build certifying a falsehood, and each was reproduced before it was fixed. The narrow lesson is that a machine-checked proof in a repository is two claims, that proof assistants have solved the first, and that the second is currently defended by scripts nobody reviews. The broader one is that the failures share a shape — *a control whose scope is chosen by the thing it controls* — for which the fix is not a longer denylist but inverting the direction of the choice. We develop both in a companion paper (Glova 2026b); here they serve as the calibration for §4’s numbers.

Our next steps are the ones the limitations dictate, in order: a policy frontend usable by an engineer who does not know SKI, because the sharpest external critique we received is that we hardened the most visible surface rather than the one a user meets first; an external implementation of Book I from the English specification

by someone who has not read our code — the single most valuable missing datum, and one no amount of internal work can supply; and an independent adversarial pass by a reviewer we did not prompt.

10. Artifact availability

The artifact is the repository:

<https://github.com/s0fractal/sigma-glyph> (Glova 2026a) commit 8d6b8234959a97be564a0301ca0b3d130c8c8c2f master

That commit carries all eleven evaluator theorems of §4.2 and the guard of §5 in its current form. All figures in §6 were measured at 35d8aea, three commits earlier; `proofs/` and `tests/proof_guard_test.py` are byte-identical between the two, which we verified with `git diff`. The most recent governance-adopted anchor set is the tag `v0.6.7` at 16a1355; §7 explains the difference.

MIT for the implementation, CC-BY-4.0 for the specification texts. The `v0.6.7` anchor set was adopted under the project’s 2-of-3 threshold policy (adoption warrant b4dc05e307b8, two valid signatures, which we verified independently); §7 states why that is a satisfied policy and not independent attestation.

Packaging. The stack’s three Python packages are published on PyPI and were installed into a clean virtual environment and run by the maintainer: `sigma-glyph` 0.6.7, `warrant-verify` 0.6.0 and `oaip` 0.3.0, all uploaded 2026-07-31 through GitHub Actions OIDC Trusted Publishing. We verified the versions, filenames and upload timestamps against the PyPI JSON API rather than against the repository’s own documentation, which at the time of writing still described earlier versions as the latest. No third party has installed them, so we claim *published and installed by the maintainer* and nothing stronger. Note also that `sigma-glyph`’s distribution ships the Book I–III Python modules only: the specification, the conformance vectors, the Lean proofs and the Rust and Go implementations live in the repository, so the artifact for this paper is the checkout, not the wheel.

Prerequisites: Python 3.10+, and `elan` for the pinned Lean toolchain (`leanprover/lean4:v4.31.0`, read from `proofs/lean-toolchain`). The Rust and Go surfaces additionally need `cargo` and `go`; the Lean surfaces need nothing else — no `mathlib`, no network.

One command reproduces everything in §6.1 and §6.2:

```
git clone https://github.com/s0fractal/sigma-glyph && cd sigma-glyph
git checkout 8d6b8234959a97be564a0301ca0b3d130c8c8c2f
tools/test-all.sh
```

`test-all.sh` prints ALL GREEN only if every surface ran. A surface that could not be checked — no `lean` on `PATH`, no network for the cross-repo parity pulls — is named in the verdict and the script exits 2. (Until 2026-07-29 it printed ALL GREEN and exited 0 after skipping, which is itself an instance of §5’s thesis; `proof_guard.py` invoked with no arguments did the same thing until 5e75e9b.) `ALLOW_SKIPS=1` is how a reader says the gap was deliberate.

To reproduce the proof results alone, offline:

```
python3 proofs/bridge_check.py           # SizeBound guard + 861 oracle steps
python3 proofs/byte_bridge_check.py      # BYTE-BRIDGE: ALL AGREE (334/334)
python3 proofs/eval_bridge_check.py      # EVAL-BRIDGE: ALL AGREE (33/33)
python3 proofs/wave_bridge_check.py      # WAVE-BRIDGE: ALL AGREE (582/582)
python3 proofs/c1_bridge_check.py        # C1-BRIDGE: ALL AGREE (3000 terms)
python3 tests/proof_guard_test.py        # PROOF-GUARD: ALL PASS (122 checks)
```

Each bridge runs the integrity guard of §5 before it runs anything else, and fails closed if `lean` is absent (exit 2, never a silent skip). Total cold wall time for the five bridges on the machine described in §6 is 42.3 s; the guard regression adds 38.9 s. Run the guard regression on a clean tree: several of its vectors write

and then remove real files under `proofs/`, including a subdirectory, as part of testing the unaudited-file and non-recursive-walk cases.

Acknowledgements

The adversarial review rounds of §5 were conducted by fresh-context language model sessions prompted by the author; the reproductions they produced are the evidence for every vector reported. Earlier design decisions recorded in ADR-001 through ADR-007 — in particular the hash-leaf size model, whose proof and independent re-derivation are cited in Book I §3.4 — came out of a multi-model review protocol documented in `reviews/`. Neither process constitutes an independent audit, and §7 says so.

The Lean formalizations, the reference implementations, and much of this prose were written by AI models — OpenAI Codex and GPT, Anthropic Claude, Moonshot Kimi, Google Gemini and Antigravity, DeepSeek, Alibaba Qwen, and Zhipu GLM — working under the direction of the author, who specified the system, stated the theorems to be proven, and curated the adversarial review process. The per-model review ledger, with dates and dispositions, is preserved in the repository under `reviews/`; the counts there are the honest record of which model found what, including the rounds that found nothing.

That authorship claim is deliberately explicit rather than buried, because the paper’s own argument is about not taking a producer’s word for its output. Every theorem is checked by the Lean 4 kernel, which does not care who wrote the tactic script; the trusted-base caveats, `native_decide` among them, are stated in `proofs/README.md` and in §7. What a model wrote and what a kernel accepted are two different claims, and only the second one carries weight here.

References

- Amani, Sidney, Myriam Bégel, Maksym Bortin, and Mark Staples. 2018. “Towards Verifying Ethereum Smart Contract Bytecode in Isabelle/HOL.” *Proceedings of the 7th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*, 66–77. <https://doi.org/10.1145/3167084>.
- Barendregt, Hendrik Pieter. 1984. *The Lambda Calculus: Its Syntax and Semantics*. Revised. North-Holland.
- Barendregt, Henk, and Freek Wiedijk. 2005. “The Challenge of Computer Mathematics.” *Philosophical Transactions of the Royal Society A* 363 (1835): 2351–75. <https://doi.org/10.1098/rsta.2005.1650>.
- Benet, Juan. 2014. “IPFS – Content Addressed, Versioned, P2P File System.” *arXiv Preprint arXiv:1407.3561*, ahead of print. <https://doi.org/10.48550/arXiv.1407.3561>.
- Carneiro, Mario. 2019. “The Type Theory of Lean.” Master’s thesis, Carnegie Mellon University.
- Carneiro, Mario. 2024. *lean4lean: A Verified Type Checker for Lean 4, in Lean 4*. <https://github.com/digama0/lean4lean>.
- Chalkias, Konstantinos, François Garillot, and Valeria Nikolaenko. 2020. “Taming the Many EdDSAs.” *Security Standardisation Research (SSR)*, Lecture notes in computer science, vol. 12529: 67–90. https://doi.org/10.1007/978-3-030-64357-7_4.
- Charguéraud, Arthur, and François Pottier. 2019. “Verifying the Correctness and Amortized Complexity of a Union-Find Implementation in Separation Logic with Time Credits.” *Journal of Automated Reasoning* 62 (3): 331–65. <https://doi.org/10.1007/s10817-017-9431-7>.
- Chiusano, Paul, Rúnar Bjarnason, and the Unison contributors. 2024. *The Unison Language*.

<https://www.unison-lang.org/>.

Curry, Haskell B., and Robert Feys. 1958. *Combinatory Logic, Volume i*. North-Holland.

Dolstra, Eelco. 2006. “The Purely Functional Software Deployment Model.” PhD thesis, Utrecht University.

Glova, Serhii. 2026a. *Σ -GLYPH: A Portable Deterministic Check Engine with Size-Priced Budgets*. <https://github.com/s0fractal/sigma-glyph>.

Glova, Serhii. 2026b. *Twenty-One Ways Past a Proof Guard: The CI Apparatus Around a Theorem Prover Is the Attack Surface*. Companion preprint.

Glova, Serhii. 2026c. *Warrant: Signed, Hash-Addressed Decision Records for AI Agents*. <https://github.com/s0fractal/warrant>.

Goldberg, Lior, Shahar Papini, and Michael Riabzev. 2021. *Cairo – a Turing-Complete STARK-Friendly CPU Architecture*. Cryptology ePrint Archive, Paper 2021/1063.

Guéneau, Armaël, Arthur Charguéraud, and François Pottier. 2018. “A Fistful of Dollars: Formalizing Asymptotic Complexity Claims via Deductive Program Verification.” *Programming Languages and Systems (ESOP)*, Lecture notes in computer science, vol. 10801: 533–60. https://doi.org/10.1007/978-3-319-89884-1_19.

Haas, Andreas, Andreas Rossberg, Derek L. Schuff, et al. 2017. “Bringing the Web up to Speed with WebAssembly.” *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 185–200. <https://doi.org/10.1145/3062341.3062363>.

Hildenbrandt, Everett, Manasvi Saxena, Nishant Rodrigues, et al. 2018. “KEVM: A Complete Formal Semantics of the Ethereum Virtual Machine.” *IEEE 31st Computer Security Foundations Symposium (CSF)*, 204–17. <https://doi.org/10.1109/CSF.2018.00022>.

Hirai, Yoichi. 2017. “Defining the Ethereum Virtual Machine for Interactive Theorem Provers.” *Financial Cryptography and Data Security (WTSC)*, Lecture notes in computer science, vol. 10323: 520–35. https://doi.org/10.1007/978-3-319-70278-0_33.

Hoffmann, Jan, Klaus Aehlig, and Martin Hofmann. 2012. “Resource Aware ML.” *Computer Aided Verification (CAV)*, Lecture notes in computer science, vol. 7358: 781–86. https://doi.org/10.1007/978-3-642-31424-7_64.

Hofmann, Martin, and Steffen Jost. 2003. “Static Prediction of Heap Space Usage for First-Order Functional Programs.” *Proceedings of the 30th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 185–97. <https://doi.org/10.1145/604131.604148>.

Josefsson, Simon, and Ilari Liusvaara. 2017. *Edwards-Curve Digital Signature Algorithm (EdDSA)*. RFC 8032, IETF. <https://doi.org/10.17487/RFC8032>.

Klein, Gerwin, Kevin Elphinstone, Gernot Heiser, et al. 2009. “seL4: Formal Verification of an OS Kernel.” *Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP)*, 207–20. <https://doi.org/10.1145/1629575.1629596>.

Kumar, Ramana, Magnus O. Myreen, Michael Norrish, and Scott Owens. 2014. “CakeML: A Verified Implementation of ML.” *Proceedings of the 41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 179–91. <https://doi.org/10.1145/2535838.2535841>.

Lean FRO. 2024. *lean4checker: An External Checker for Lean 4 Environments*. <https://github.com/leanprover/lean4checker>.

- Leroy, Xavier. 2009. “Formal Verification of a Realistic Compiler.” *Communications of the ACM* 52 (7): 107–15. <https://doi.org/10.1145/1538788.1538814>.
- McKeeman, William M. 1998. “Differential Testing for Software.” *Digital Technical Journal* 10 (1): 100–107.
- Moine, Alexandre, Arthur Charguéraud, and François Pottier. 2023. “A High-Level Separation Logic for Heap Space Under Garbage Collection.” *Proceedings of the ACM on Programming Languages* 7 (POPL): 718–47. <https://doi.org/10.1145/3571218>.
- Moura, Leonardo de, and Sebastian Ullrich. 2021. “The Lean 4 Theorem Prover and Programming Language.” *Automated Deduction – CADE 28*, Lecture notes in computer science, vol. 12699: 625–35. https://doi.org/10.1007/978-3-030-79876-5_37.
- National Institute of Standards and Technology. 2015. *Secure Hash Standard (SHS)*. FIPS PUB 180-4. <https://doi.org/10.6028/NIST.FIPS.180-4>.
- OpenSSF. 2025. *SLSA: Supply-Chain Levels for Software Artifacts*. <https://slsa.dev/>.
- Protocol Labs. 2024. *Multiformats: Self-Describing Values for Future-Proofing*. <https://multiformats.io/>.
- Ringer, Talia, Karl Palmskog, Ilya Sergey, Milos Gligoric, and Zachary Tatlock. 2019. “QED at Large: A Survey of Engineering of Formally Verified Software.” *Foundations and Trends in Programming Languages* 5 (2–3): 102–281. <https://doi.org/10.1561/25000000045>.
- RISC Zero. 2025. *RISC Zero ZkVM*. <https://github.com/risc0/risc0>.
- Rundgren, Anders, Bret Jordan, and Samuel Erdtman. 2020. *JSON Canonicalization Scheme (JCS)*. RFC 8785, IETF. <https://doi.org/10.17487/RFC8785>.
- Schönfinkel, Moses. 1924. “Über Die Bausteine Der Mathematischen Logik.” *Mathematische Annalen* 92: 305–16. <https://doi.org/10.1007/BF01448013>.
- Succinct Labs. 2025. *SP1: A Performant, Open-Source Zero-Knowledge Virtual Machine*. <https://github.com/succinctlabs/s>.
- The mathlib Community. 2020. “The Lean Mathematical Library.” *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*, 367–81. <https://doi.org/10.1145/3372885.3373824>.
- Torres-Arias, Santiago, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, and Justin Cappos. 2019. “In-Toto: Providing Farm-to-Table Guarantees for Bits and Bytes.” *28th USENIX Security Symposium*, 1393–410.
- Turner, David A. 1979. “A New Implementation Technique for Applicative Languages.” *Software: Practice and Experience* 9 (1): 31–49. <https://doi.org/10.1002/spe.4380090105>.
- W3C WebAssembly Working Group. 2025. *WebAssembly Core Specification*. <https://webassembly.github.io/spec/core/>.
- Watt, Conrad. 2018. “Mechanising and Verifying the WebAssembly Specification.” *Proceedings of the 7th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*, 53–65. <https://doi.org/10.1145/3167082>.
- Wiedijk, Freek. 2012. “Pollack-Inconsistency.” *Electronic Notes in Theoretical Computer Science* 285: 85–100. <https://doi.org/10.1016/j.entcs.2012.06.008>.

- Wood, Gavin. 2014. *Ethereum: A Secure Decentralised Generalised Transaction Ledger*. Yellow Paper. Ethereum Project.
- Yang, Xuejun, Yang Chen, Eric Eide, and John Regehr. 2011. “Finding and Understanding Bugs in C Compilers.” *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 283–94. <https://doi.org/10.1145/1993498.1993532>.
- Yarvin, Curtis, Philip Monk, Anton Dyudin, and Raymond Pasco. 2016. *Urbit: A Solid-State Interpreter*. <https://media.urbit.org/whitepaper.pdf>.
- Zinzindohoué, Jean-Karim, Karthikeyan Bhargavan, Jonathan Protzenko, and Benjamin Beurdouche. 2017. “HACL*: A Verified Modern Cryptographic Library.” *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 1789–806. <https://doi.org/10.1145/3133956.3134043>.